
AI-Enhanced F1 Simulator

Individual Report

Author:
JUNJIAN CAO

Supervisor:
RUZANNA CHITCHYAN

Department of Engineering

UNIVERSITY OF BRISTOL

An individual report submitted to the University of Bristol as part of the requirements of the degree of MASTER OF SCIENCE, describing my personal contribution to the group project *AI-Enhanced F1 Simulator*.

SEPTEMBER 2026

Word count: approximately 5,600 (body text, Sections 1–6)

Abstract

This report describes my part of the group project *AI-Enhanced F1 Simulator*. I worked on the Live Commentary feature with my teammate, Songyun Han. This is one of four AI features in the project.

I was mainly responsible for the part of the system that comes after the simulator detects a race event. I turned each selected event into a prompt for the model. I also controlled how the speech and captions were played, so that a busy moment in the race did not cause silence or overlapping speech. I built the tests used to check that this actually worked.

After a short introduction, my report has five main parts. First, I explain what I did in this project, and the ideas and new approaches behind the feature design. Second, the report describes how I built these features. This includes how I decided, when the system was busy, whether a new event should preempt the one already queued or be dropped. It also covers how I designed the prompts. Third, I set out how I tested the Commentary feature, and the methods I used to check that it worked correctly and reliably. Fourth, the report outlines how my teammate and I planned our work, including how we split tasks and scheduled the different phases of the work. Finally, I critically evaluate the work and look ahead to future work.

Author's Declaration

I declare that the work in this report was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in this report are those of the author.

SIGNED: DATE:

Table of Contents

1	Introduction and Context	1
2	Innovation and Creativity	2
3	Technical Development	3
3.1	Diagnosing the problem	4
3.2	First iteration: silent prefetch	4
3.3	Second iteration: a queued slot with priority replacement	4
3.4	Field-tagged prompt formatting	5
3.5	A related defect: context-budget headroom	6
4	Evaluation Skills	6
5	Project Management Skills	8
5.1	Fitting personal planning into the team’s cadence	8
5.2	Personal task sequencing	8
5.3	Working within the group’s interfaces	8
6	Critical Self-Evaluation and Future Work	9
6.1	Withdrawing bilingual commentary	9
6.2	The single-highest-priority-per-cycle limitation	9
6.3	Naming as an interface	10
6.4	Future work	10
A	Code Figures	12

1 Introduction and Context

Our group project is called *AI-Enhanced F1 Simulator*. It is a racing simulator with AI features, built on top of TORCS [1], an open-source driving simulator that supplies real-time data. Like any real-time racing simulator, TORCS produces telemetry much faster than a user can read it while driving. This is because vehicle position, damage, lap time, and the distance to nearby cars all update many times each second. In real motorsport, a broadcast commentator does a similar job. The commentator takes in far more information than a spectator has time to follow. They pick out the small part that really matters, and turn it into a sentence for spectators. This helps spectators quickly understand what is happening on track. In this project, the Live Commentary module plays this role. I worked on it together with my teammate, Songyun Han.

This feature serves two groups of users: drivers and spectators. A driver needs to keep almost all of their attention on the simulator screen. Commentary text on screen would pull this attention away, so we chose a channel that does not compete for visual attention: speech. With voice output, a spoken line about a rival closing from behind, or about the driver's own position changing, lets the driver understand how the race has changed. They never need to look away from the track [3]. For a spectator watching someone else drive, or watching the AI Driving Bot race on its own, the feature acts like the commentator a real broadcast would provide. This is something none of the project's other three features attempt to do. Using both text and speech together meets spectators' need to follow information that changes moment to moment. Neither drivers nor spectators want a system that talks constantly or repeats itself. This is why, as later sections describe, a large part of my own work was not about generating text. It was about selecting and pacing it.

Live Commentary is one of three features that need the same real-time simulator data. The other two are the Race Engineer and the Telemetry Coach. Instead of each feature opening its own connection to TORCS, the project built one shared layer to collect this data. This telemetry arrives through a read-only UDP interface. A single `TelemetryService` parses it and stores it in a thread-safe `TelemetryStore`. Any of the three features can read from this store without competing for the same socket. Live Commentary is built directly on top of this shared store. Everything downstream of it, however, is private to Commentary: event detection, prompt construction, and output scheduling. This part of the system was built only for narration, not for diagnosis or advice. The one place where private content passes back through shared infrastructure is the model call itself. This call goes through the project's shared `ModelBroker`. My own contribution covers this private layer. I explain how a selected race event becomes a bounded prompt, how its output is scheduled so it does not fall silent or talk over itself, and how I tested this pipeline.

Figure 1 shows this structure from end to end: the shared infrastructure at the top; in the middle, the point where Commentary's own payload construction diverges from how the other two telemetry-fed features build their prompts; and the private output-scheduling pipeline at the bottom. Section 3 covers this pipeline in detail.

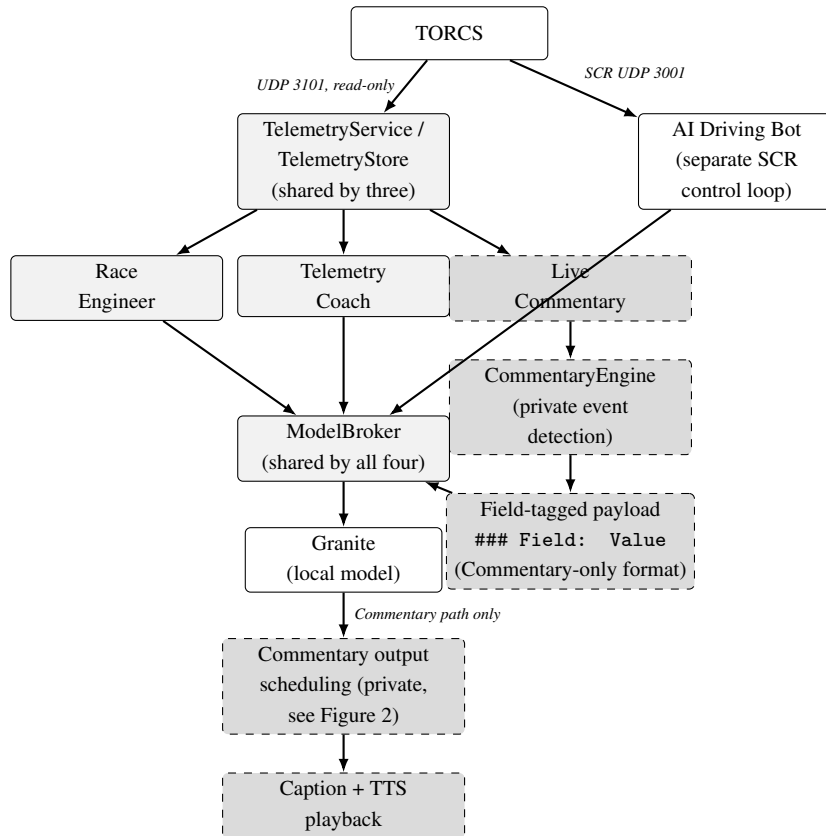


Figure 1: Shared workflow (solid boxes) versus Commentary’s own private workflow (dashed boxes). Telemetry acquisition is shared by three features; the model broker is shared by all four. SCR (Simulated Car Racing) is the TORCS control interface used by the AI Driving Bot.

The AI Driving Bot’s strategy really does change how the car is driven. Commentary, however, is different: its model output never enters the control loop. At worst, a slow or unavailable model call means one race event goes unnarrated. It does not mean the simulation breaks. Commentary only watches the race. It never controls the car. This limited role is the main reason the feature matters. It makes it reasonable to put a language model between a fast-running simulation and its user. A language model’s response time is measured in seconds, and its output cannot be fully guaranteed. The same bounded, watch-only role could apply beyond racing: wherever a real-time system produces data faster than a person can follow, a language model can add a human-readable layer without ever being trusted with control. This could work for live sports data, industrial monitoring, or a real-time dashboard, for example. Sections 3 and 4 look at this boundary more closely. They describe how the system stops a pending or delayed model response from ever blocking the underlying telemetry pipeline. I also built the tests to check that this holds, under both real faults and faults I created on purpose.

2 Innovation and Creativity

On the Commentary feature, Han and I split the work by stage: he was responsible for the early architecture, and I was responsible for the feature work built on top of it. Han built the part that decides whether a telemetry change is worth narrating. It has three pieces: a sliding-window event detector, a priority ranking for different event types, and cooldown and deduplication rules. These rules stop the same battle or the same off-track moment from being narrated again on every telemetry tick. In short, it solves one question: is this event worth narrating? Section 4 later confirmed that it works well, drawing on both the test plan my teammate prepared

and my own checks against real driving footage.

But this design does not solve another question: once an event is judged worth narrating, is it still worth playing right now? As soon as this happens, Commentary asks Granite for a line. However, a more important event can appear while that first line has not yet finished generating or playing. In an early version of the pipeline, the same piece of logic handled both questions. In other words, the system treated “worth narrating” and “worth playing right now” as the same decision. This worked fine when commentary was rare. Yet it broke down exactly when commentary mattered most: during a multi-car battle, or a lap full of contact and off-track recoveries. In these moments, several events can all be worth narrating within the few seconds it takes to generate and play just one line.

I noticed this problem while watching an early demonstration, and raised it with Han. Watching the recording together, we agreed that “worth narrating” and “worth playing right now” were not really the same question. We talked through a few ways to fix it, and decided together that Commentary needed a separate output-scheduling layer: two smaller state machines connected by a simple interface, instead of one bigger machine. I designed and built this layer myself, as an interrupt/queue playback mode, sitting entirely downstream of Han’s event/interval/hybrid trigger logic. This let me reason about, and test, two questions on their own: “is this worth narrating,” and “what happens to speech that is already queued when something more important arrives.” Section 3 explains this scheduler in more detail. The key creative step here was recognising that what looked like a single decision was actually two separate questions. This was not obvious at the time: as described above, the shared logic worked well until commentary mattered most, and only broke down under real racing conditions. At the time I built the second state machine, only its symptoms existed – playback going silent or badly timed when several events happened close together.

There is a smaller example of the same idea: not accepting something just because it already exists. It concerns how a selected event reaches the model. When we first built this part, Han and I simply wrote the event payload as JSON, and sent it to the model as it was. We did not think much about the format itself. Later, we discussed it, and realised this choice was probably a problem. It affected the token cost of every request, and the quality of what the model generated. JSON’s brackets, quotes, and indentation do not describe the race at all. But the model still has to read them, and sometimes even repeats them back in its own output. Once we saw this as a real problem, not just a guess, I searched for how other people had approached the same task: turning structured data into a prompt. I found a published system that used a similar convention for a closely related task. Section 3 discusses this in more detail. I adopted the same idea here: a plain-text format with field labels instead of JSON. For example, `### Speed change: +38 km/h` instead of `"speed_delta": 38`. This format only includes the fields that matter for each event type, instead of sending the same full payload every time.

3 Technical Development

This section gives more detail about the parts of the Commentary feature I built myself – mainly the output scheduler and the prompt-building around it – than the group report did. It covers, first, why I built the scheduler and how its design changed over two iterations; second, the field-tagged prompt format I adopted for building each request; and finally, a context-budget defect I found during testing, and how I fixed it.

3.1 Diagnosing the problem

This subsection first sets out where the scheduler sits in the pipeline, and what it must do. Upstream, event selection decides whether a telemetry change is worth narrating; this is the part Han built, described in Section 2. Downstream, the scheduler must keep two outputs in sync: the caption shown on the shared dashboard, and the spoken line played through the text-to-speech (TTS) service. Its job is this: once it receives a selected event, it must make sure the caption text and the spoken commentary appear together.

In an early, single-slot version, the scheduler could hold only one pending request at a time. A new event either waited for the current request to finish, or, in interrupt mode, cancelled it and started generating a new one from scratch. This second behaviour caused the problem. When the scheduler cancelled a request and started a new one, the dashboard showed no caption and the TTS service played no audio, for however long Granite took to produce the first token of the new line. A burst of two or three events close together is exactly the moment when commentary matters most. In this situation, the scheduler did not produce faster commentary. Instead, it produced a gap with no caption and no audio, followed by a single line describing only the last event.

3.2 First iteration: silent prefetch

My first fix mainly targeted the silence problem, which happened whenever the scheduler cancelled the current request and started generating a new one from scratch, leaving the dashboard with no caption and the TTS service with no audio until the new line was ready. Instead of cancelling the in-flight request outright, a new, higher-priority event triggered a second, “silent” generation request to the `ModelBroker` straight away. The line already playing was allowed to finish, instead of being cut off mid-sentence. “Silent” here meant the request ran in the background, and its tokens built up there. Nothing was sent to the `WebSocket` or the TTS service until it was that request’s turn. This meant the dashboard and the TTS service were never left empty while a new line was still generating, but it created a new problem. With only one pending slot, a third event arriving while the silent request was still generating had nowhere to go, except to cancel it. This brought the original problem back one level up, now for bursts of three events instead of two. Figure 3 shows the function this silent request – the prefetch – became: `_start_prefetch()` in `middleware/runtime.py`. It cancels any earlier prefetch that is still running, before starting a new one for whatever event is now queued.

3.3 Second iteration: a queued slot with priority replacement

The final version does not cancel the line already playing at all. Module-level state in `middleware/runtime.py` tracks two things separately: `_commentary_task`, the coroutine that is currently generating or already playing, and `_pending_decision`, at most one event waiting to take over once the current line finishes. A new event that arrives while the system is busy goes through `_queue_event()` (Figure 4). If its priority is lower than whatever is already in `_pending_decision`, it is dropped outright. Otherwise, it replaces the pending slot, and `_start_prefetch()` is called again, so the silent generation stays in sync with whatever is now pending. This design – where a lower-priority event is discarded, rather than entering a queue – was not something I decided alone. Following our supervisor’s suggestion to use a queue (Section 4), Han and I determined this design together, through discussion, after testing an earlier version. We noticed that, during busy periods, a queue that accepted every collected event fell further and further behind the live race. We agreed that dropping a lower-priority event was better than the scheduler eventually describing something that had already happened several seconds earlier.

This kind of problem has its own name in the systems literature: load shedding. It means deciding what to discard when a system cannot keep up with everything arriving at it. After Han and I had settled on dropping rather than queuing, I looked for how this problem had been studied elsewhere. I found Tatbul et al.’s work on load shedding in a data stream manager [7]. Their system faces a similar choice at a much larger scale. When an overloaded stream processor cannot keep up with every incoming tuple, it must decide what to discard. Their answer is to drop from whichever output stream’s quality would degrade the least, rather than discarding at random or spreading the loss evenly across every stream. My own `_queue_event()` logic is, in essence, a small-scale drop decision: instead of treating every queued event as equally worth keeping, it keeps the more important one and drops the less important one. The event priorities are what make this a principled drop rather than an arbitrary one – the same reasoning behind Tatbul et al.’s choice to shed load where quality degrades least.

When the front end reports that playback has finished, `_advance_queue()` (Figure 5) releases the pending slot. If the system is not in queue mode, or nothing is pending, it simply clears the busy flag and goes idle. Otherwise, it takes over the pending decision and its prefetch task. It waits for that prefetch to finish, if it has not already, before broadcasting the result as the next caption and audio clip, through `_broadcast_silent_result()`.

Figure 2 sets out the finished pipeline described above, from a selected event to the caption and audio that eventually reach the user.

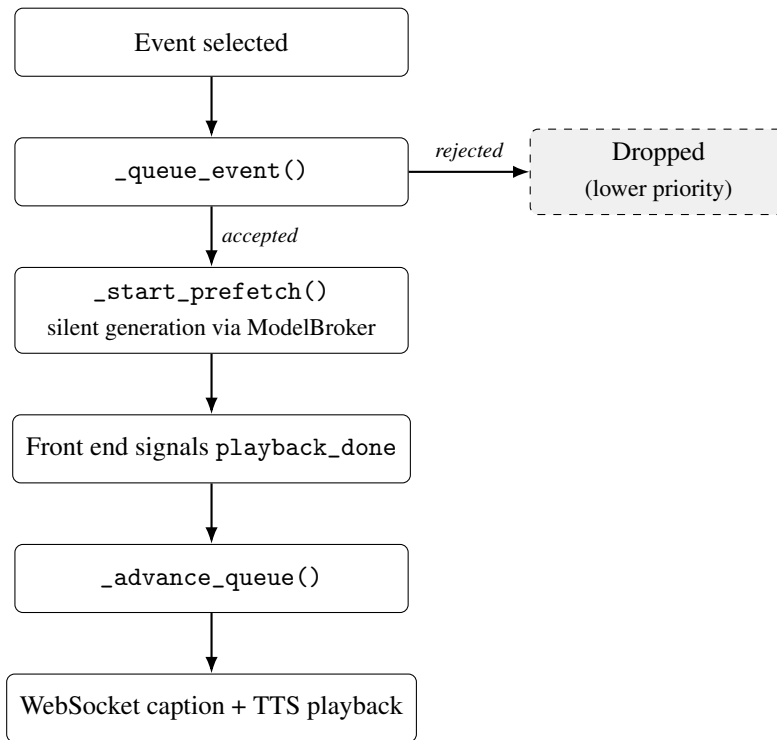


Figure 2: The finished output-scheduling pipeline, from a selected event to the caption and audio delivered to the user. The dashed box marks the one point where an event is discarded rather than carried forward.

3.4 Field-tagged prompt formatting

The field-tagged rendering described in Section 2, which writes each piece of event data as `### Field: Value` rather than as JSON, is something Han and I arrived at together, through reflection after testing. We looked into how large language models process input and how they consume tokens, and found that the original

JSON format was probably costing us both tokens and generation quality. I then looked for how other people had solved a similar problem, and found Cook and Karakuş’s paper on fine-tuning a language model to generate football commentary from structured match-event data [6]. Their system also formats each piece of information fed to the model with a `###` prefix, specifically so the model can reliably tell where one piece of event data ends and the next begins. This is what led me to adopt the same approach here, rather than inventing something from scratch.

This rendering method is used differently in this project than in the cited literature. Their prefix marks up training examples that get baked into a model’s fine-tuned weights. Mine marks up a payload that is assembled fresh for every request, against a model that is not fine-tuned for Commentary at all. Because this project could not fine-tune the Granite model, the same formatting idea had to do a different job here. Instead of teaching the model a pattern during training, it has to make that pattern clear to the model at inference time, on top of whatever the model already knows about reading structured text.

3.5 A related defect: context-budget headroom

This context-budget defect was not discovered while I was thinking about the design. It was found during one of my manual test rounds (Section 4), when a request that should have worked was rejected instead. The `ContextManager` builds each request’s prompt out of the persona, the event payload, and a limited slice of recent commentary history. It estimates how many tokens this text will cost using a quick heuristic: about one token per four ASCII characters, and one token per two CJK (Chinese, Japanese, and Korean) characters. It does not use the model’s real tokenizer, because running a real tokenizer on every frame would be too slow on a locally hosted model. This heuristic is only ever a rough guess. During a long test session, it sometimes underestimated a large history slice by just enough to push the prompt over the model’s real limit. When that happened, the request was rejected straight away, instead of being trimmed further.

The first version of the budget left no room for this kind of mistake. The token budget for history was calculated as exactly `max_context_tokens` minus `max_response_tokens`, with nothing left over if the estimate was wrong. This was an easy mistake to make the first time round, because when it failed, it looked like a model problem rather than a budgeting one. My fix was small: a fixed safety margin (`safety_margin_tokens`) is now subtracted from the history budget before trimming. This means the heuristic can be a little bit wrong, without the request failing. I reproduced this failure on purpose, in a later test round, to make sure the fix actually worked, instead of just assuming it did.

In retrospect, these two pieces of work – the queued-slot scheduler and the context-budget fix – between them capture the two kinds of problem I kept running into on the Commentary feature. One was architectural, caused by two jobs sharing one mechanism. The other was a mistake in how the code around a heuristic handled its error. The group report’s higher-level description of the pipeline does not cover either of these problems in detail. This is why I wanted to explain them properly here.

4 Evaluation Skills

My teammate, Han, wrote the test plan for Commentary, which set out the event categories to cover, and the pass or fail criteria for detection accuracy and priority behaviour. My job was to run this plan against the finished implementation, find where it broke, and fix the code. To do this, I wrote the test code myself: mostly a set of automated checks, each one feeding a hand-built telemetry frame into the detector and confirming

that it returned the correct event type. This meant I could re-run the checks in seconds after a code change, instead of driving a full session every time. I also wrote a monitoring script that logged every event the system reported, with a timestamp, and a separate script that matched those logs against the events I had labelled by hand, to work out the detection accuracy. On top of this, I checked for myself that what the system said it detected actually matched what happened while driving, instead of just trusting the log. This mattered because a language model can produce a line that sounds reasonable, even when it is not grounded in what the telemetry actually showed [2].

Evaluation of this feature did not only come from my own test plan. One design decision came directly from formative feedback from our supervisor. Near the end of the project, as we were preparing for our planned user evaluation, our supervisor asked us to demo the feature during a group meeting. She found the immediate-interruption delivery jarring. She did not like how a new event cut off the line already playing, and she suggested handling short bursts of events with a queue instead. This prompted me to redesign the mechanism, away from immediate interruption and towards queuing new lines instead of cutting off the current one. Once I started building it, though, I found that a queue accepting every event fell behind the live race during busy periods. So I built a single-slot mechanism with priority-based replacement instead of queuing every event; the choice to drop lower-priority events rather than queue them was one Han and I reached together, as Section 3 describes. This was formative feedback driving a design iteration, not a result from our planned user evaluation.

I ran through the whole plan by hand four times during development, not just once at the end. In each of the first three rounds, I found a real problem. One was a duplicate commentary line, caused by the deduplication check running after the WebSocket broadcast instead of before it. Another was a badly tuned `pace_surge` threshold that fired on ordinary acceleration. The third was the context-budget rejection I described in Section 3. Each time, I traced the problem back to the exact piece of code causing it, fixed it, and re-ran that part of the plan before moving on. Only in the fourth round did everything in the plan pass cleanly. That was when I considered the feature ready, against the criteria Han had set.

The part of this process I put the most personal effort into was checking that a passing result was actually true, not just consistent with itself [4]. A detection script can log a sequence of events that looks fine, even when there is, in fact, a problem with it. For example, it might log a contact event but miss an off-track excursion that happened at the same time. Nothing in the log reveals this on its own. To check for this, I recorded precise start and end timestamps for every test drive, and screen-recorded the whole session. Afterwards, I went through the CSV file from the monitoring script entry by entry, using the timestamps to match each logged event to the exact moment in the recording. This is how I confirmed, instead of just assumed, that the events the script reported actually matched what happened on screen. It also let me find a real limitation that the log alone would never have shown. When an off-track excursion happened in the same cycle as a contact event, the recording clearly showed both. But the detector keeps only one match per cycle, and the two events shared the same priority, so one was silently dropped: only the contact was logged.

Taken together – passing every criterion in Han’s test plan, being verified against real footage rather than a self-consistent log, and holding under both real faults and faults I created on purpose – this gives me confidence that the feature is of high quality. I discuss the priority-collision limitation above, and what I would do differently about it, in Section 6.

5 Project Management Skills

The group report already describes project management at the team level. This section covers how I personally planned, managed, and took part in developing the Commentary feature with Han, and how this two-person, collaborative way of working shaped my own methods.

5.1 Fitting personal planning into the team’s cadence

Our development followed the SCRUM methodology [5]. Each week was run as a sprint. The team met with our academic supervisor, Ruzanna Chitchyan, every Monday morning, to report on the previous week and get feedback on direction. Straight after that, the whole team held an internal planning meeting on the same Monday afternoon. This meant task assignment for the coming week could reflect the supervisor’s latest comments, instead of lagging behind them. It was in that Monday-afternoon meeting that Han and I decided, week by week, who would take which part of Commentary’s work. We logged the results on the shared Jira board the whole team used, rather than keeping a private list just for the two of us. Every Friday, we also sent a written progress update by email to both our academic and IBM supervisors. I had to plan my own weekly work carefully, so that I always had something worth reporting at the group meeting. This let me get feedback from our supervisor in good time, and use it to improve my own design.

5.2 Personal task sequencing

Inside that weekly structure, I built my own work in layers, instead of trying to build the whole commentary pipeline in one go. Each layer had to work end to end before I added anything more complicated on top of it. I first built a small middleware service and a `ContextManager` that could take live telemetry and produce one spoken line, along with a basic caption and voice front end. This gave the team something working within the first two weeks, even though every part of it was still rough. Only after that did I go back and improve the parts that mattered for quality: the prompt design and the context budget, and eventually the interrupt/queue scheduling layer from Section 3, once the earlier, simpler approach could not keep up with how often events happened during a real race. I used Git branches inside our shared Commentary feature branch (`voice2.0`, `voice6.0`, `commentary4.0`, and so on) to keep these changes separate. This meant an experiment with the prompt or the playback logic would not break a version that Han, or the rest of the team, was already relying on. It also matched the team’s own rule: changes only get merged into the branch everyone else sees once they have been tested on more than one machine.

5.3 Working within the group’s interfaces

Commentary depended on infrastructure that other people owned: Han’s event-detection logic, the shared `TelemetryService`, and the shared `ModelBroker` used by every feature. Because of this, a big part of managing my own work was keeping track of changes to those upstream parts, rather than working from a specification fixed at the start of the project.

One example came up while I was refactoring `ContextManager`. I wanted to rename the `commentator_persona` field to something clearer. I decided not to. That exact name was already part of the API used by the dashboard configuration panel that Shangqing Li had built for the Race Engineer. Renaming it would have broken part of a system I did not own, and could not easily test myself. So I kept the name, and improved the code around

it instead. This is a small example, but I think it is a good one: treat any interface another teammate's code depends on as fixed. Even when I can see a better approach myself, I should still discuss it with the teammate first, before making a change.

The same thing happened at the output end. My caption and voice front end, and later a separate Electron overlay, were eventually replaced. This happened once the team agreed on one shared HTML dashboard for all four features. With that in place, I moved my playback logic into it, and deleted the old, separate Commentary page, instead of keeping both around. Managing my own work here meant noticing when something I had built myself had been replaced by a bigger piece of team work. It also meant choosing to migrate cleanly, rather than defend my own component.

6 Critical Self-Evaluation and Future Work

Looking back, three things from this project are worth reflecting on properly. The first is withdrawing bilingual commentary, a decision Han and I made together after finding a problem and testing it ourselves. The second is the single-highest-priority-per-cycle limitation, which only surfaced during testing (Section 4) and traces back to a design choice I accepted without questioning it. The third is a smaller, ongoing naming problem across the pipeline, which better coordination early on could have avoided.

6.1 Withdrawing bilingual commentary

At first, Commentary offered two language options for its spoken narration: English and Chinese. But the Chinese half, both the text and the audio, was not good enough. Chinese commentary often had errors that changed the meaning, or used the wrong words. At the same time, the Chinese TTS voice sounded unnatural, and was far behind the quality of the English voice. After discussing this with Han, we agreed that neither of us was happy with the design. We also showed it to the team, and the feedback was consistently poor too. So we dropped it, and kept only the English version. If bilingual, or multilingual, commentary is developed further in future, I would first set up a formal translation-quality test. I would also actively try new voice packs, in Chinese or other languages, once better ones become available.

6.2 The single-highest-priority-per-cycle limitation

Section 4 described how checking test recordings against the detection log found a case where an off-track excursion happened in the same cycle as a contact event. The detector keeps only one match per cycle, and these three event types – `contact`, `off_track` and `position_change` – all carry the same priority, so the comparison cannot separate them. Which one survives is decided not by priority but by an undocumented, order-dependent tie-break in the detection code, which consistently keeps `contact` and drops `off_track`. This lowers off-track recall precisely when both happen together. Our own test code even acknowledges this: a comment notes that it “pins the actual undocumented tie-break policy rather than assuming one.” I treated it as a priority rule, and never checked that the priorities were in fact equal, or that the real tie-break was undocumented. It was only through testing that I discovered some of its costs. The event log this design produces cannot be treated as a complete record whenever two things happen at once, and that is common during exactly the kind of aggressive driving that makes commentary worth having. If I built this again, I would separate what gets logged from what gets said out loud much earlier. I would keep every detected event

in a structured log, even when only one of them gets narrated in a given cycle, so that testing does not depend on the same single-slot decision that scheduling does. The group report's future-work section suggests the same direction for Commentary, and it is what I would work on first if I had more time.

6.3 Naming as an interface

A smaller problem that kept coming up was naming. Han's trigger modes are called `event`, `interval` and `hybrid`. The output-scheduling modes I built are called `interrupt` and `queue`. Both sets of names make sense on their own. But anyone reading the code without this report has no way to know that they describe two different layers of the same pipeline, not two competing versions of the same thing. More than once during integration, a teammate asked me which of the two settings controlled which part of the behaviour. In hindsight, agreeing on a shared naming convention with Han, before either set of names was decided, would have cost almost nothing at the time. This would have made the two-layer structure clear from the code itself, instead of only from a report written afterwards. For example, we could have used a shared prefix for trigger settings, and a different one for scheduling settings.

6.4 Future work

Beyond what I have already mentioned, I see two clear next steps for Commentary. The first is a visualisation window built directly into the simulator, to display the caption there, instead of only on the separate shared dashboard. The second builds on this: an automated test script. Using screen image recognition on this embedded window, the script could automatically check the events it recorded against what actually happened, instead of relying on the manual timestamp-and-recording check from Section 4. If new feature modules are added to the project in future, this kind of automated check would make testing far more efficient.

References

- [1] B. Wymann, E. Espié, C. Guionneau, C. Dimitrakakis, R. Coulom, and A. Sumner, “TORCS, The Open Racing Car Simulator, v1.3.5,” 2013.
- [2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023, doi: 10.1145/3571730.
- [3] M. R. Endsley, “Toward a theory of situation awareness in dynamic systems,” *Human Factors*, vol. 37, no. 1, pp. 32–64, 1995, doi: 10.1518/001872095779049543.
- [4] J. S. Santillana, “Precision is not faithfulness: Coverage-aware evaluation of grounded generation with a complete oracle,” arXiv preprint arXiv:2606.09376, 2026.
- [5] A. Srivastava, S. Bhardwaj, and S. Saraswat, “SCRUM model for agile methodology,” in *2017 International Conference on Computing, Communication and Automation (ICCCA)*. IEEE, 2017, pp. 864–869.
- [6] A. Cook and O. Karakuş, “LLM-Commentator: Novel fine-tuning strategies of large language models for automatic commentary generation using football event data,” *Knowledge-Based Systems*, vol. 300, art. 112219, 2024, doi: 10.1016/j.knosys.2024.112219.
- [7] N. Tatbul, U. Çetintemel, S. Zdonik, M. Cherniack, and M. Stonebraker, “Load shedding in a data stream manager,” in *Proceedings 2003 VLDB Conference*, Morgan Kaufmann, 2003, pp. 309–320.

A Code Figures

```
async def _start_prefetch(decision: CommentaryDecision) -> None:
    ...
    global _prefetch_task
    if _prefetch_task and not _prefetch_task.done():
        _prefetch_task.cancel()
    t, r = telemetry_store.latest()
    request_id = str(uuid.uuid4())

    async def _run():
        return await generate_commentary(
            t, r, event_payload=decision.payload, history_mode="summary",
            request_id=request_id, silent=True,
        )

    _prefetch_task = asyncio.create_task(_run())
```

Figure 3: `_start_prefetch()`, `middleware/runtime.py`: cancels any in-flight prefetch and starts a new silent generation (`silent=True`) for the currently queued event.

```
async def _queue_event(decision: CommentaryDecision) -> None:
    ...
    global _pending_decision
    new_priority = decision.event.get("priority", 0)
    if _pending_decision is not None and new_priority < _pending_decision.event.get("priority", 0):
        return
    _pending_decision = decision
    await _start_prefetch(decision)
```

Figure 4: `_queue_event()`, `middleware/runtime.py`: a lower-priority event is dropped; anything else replaces the single pending slot and re-triggers prefetch.

```
async def _advance_queue() -> None:
    ...
    global _pending_decision, _prefetch_task, _commentary_task, _commentary_priority, _playback_busy

    if commentary_engine.config.interrupt_mode != "queue" or _pending_decision is None:
        _playback_busy = False
        return

    decision = _pending_decision
    _pending_decision = None
    prefetch = _prefetch_task
    _prefetch_task = None
    _commentary_priority = decision.event.get("priority", 0)

    async def _release():
        try:
            result = await prefetch if prefetch is not None else None
        except asyncio.CancelledError:
            result = None
        except Exception as e:
            log.warning(f"Queued commentary prefetch failed: {e}")
            result = None
        if result is not None:
            await _broadcast_silent_result(result, event=decision.event, payload=decision.payload)

    _commentary_task = asyncio.create_task(_release())
```

Figure 5: `_advance_queue()`, `middleware/runtime.py`: releases the pending slot once the front end signals `playback_done`, waiting on its prefetch task if it has not finished yet.